8-Bit Linear Phase FIR Filter (Part 2)

Devansh Modi (202304009)

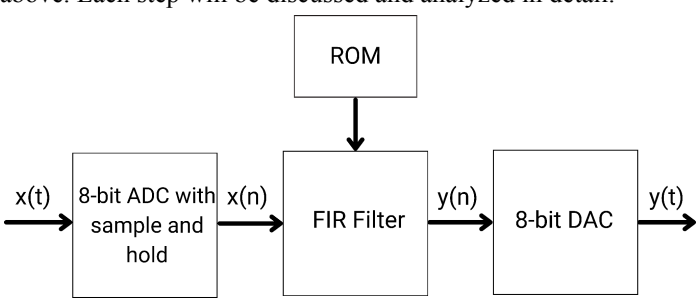
Abstract—In this assignment, the design and analysis of an 8-bit linear phase finite impulse response (FIR) low-pass filter is presented. The work includes detailed steps of filter specification, structural selection and finite wordlength analysis. The analysis shows that coefficient quantization has an insignificant effect on the filter's frequency response, ensuring accuracy and stability. The linear phase structure enhances computational efficiency while reducing hardware complexity and power consumption. Overall, the designed FIR filter provides a reliable and optimized framework for digital signal processing applications.

I. Introduction

The design of a filter involves 5 steps:

1. Filter Specification
2. Coefficients calculation
3. Filter Structure
4. Analysis of finite wordlength effects
5. Implementation

The implementation of the filter will be done in part 2 of this assignment. We will design the filter step by step as mentioned above. Each step will be discussed and analyzed in detail.



(Fig.1. A Block diagram of our digital FIR filter)

Fig.1. Shows the block diagram of our FIR filter. The analog input signal x(t) is first converted into a digital signal x(n) using an 8-bit ADC with sample-and-hold. The input signal will be sampled with a sampling frequency of 4 kHz. The FIR Filter processes the sampled data using coefficients stored in ROM, producing the digital output y(n). Finally, an 8-bit DAC converts y(n) back into an analog signal y(t).

The coefficients and input samples will be quantized by using the fi object in MATLAB. The 'fi' object in MATLAB uses signed 2's complement fixed point representation (Q7) for storing values.

II. Filter Specification

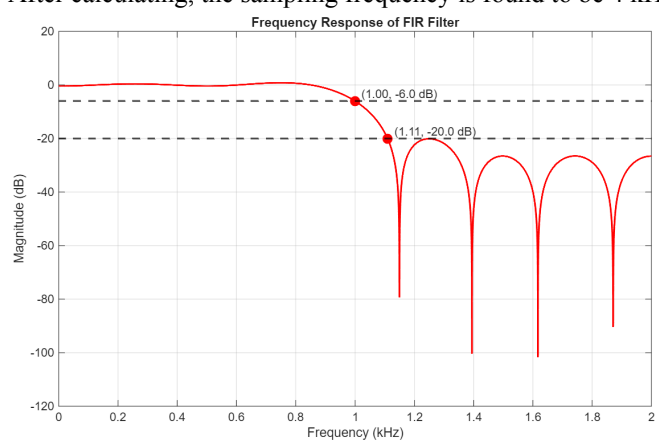
We have to design a linear phase high-performance low-pass filter. The coefficients are already given to us, and they are symmetric. The parameters given are as follows:

- ➔ Cutoff frequency (-6 dB): 1 kHz
- ➔ Stopband edge frequency: 1.11 kHz
- ➔ Stopband attenuation: >20 dB
- ➔ Coefficients, input & output samples are 8-bit wide

For digital filters, MATLAB designs filters using a normalized cutoff frequency (fn = 0.5 Hz). To find the sampling frequency (fs), the formula is:

$$F_S = 2 . \left(\frac{F_C}{F_N}\right)$$; F_N = Normalized cutoff frequency

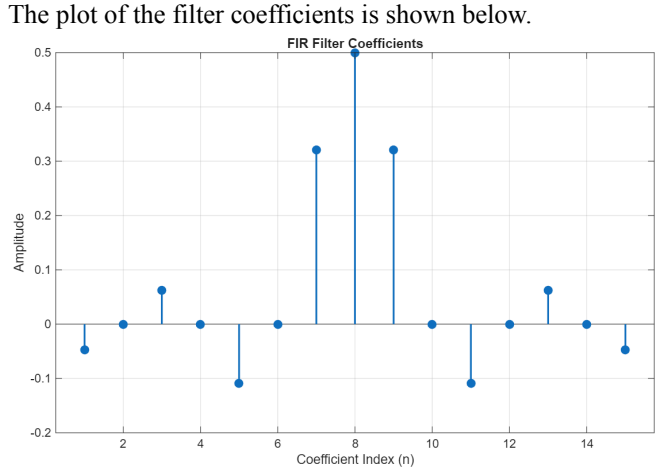
After calculating, the sampling frequency is found to be 4 kHz.



(Fig. 2. Frequency Response of FIR filter)

III. Coefficient calculation

The plot of the filter coefficients is shown below.



(Fig. 3. Filter Coefficients of the FIR filter)

IV. Analysis of finite word length effect

The problems caused by using a finite number of bits are referred to as finite wordlength effects, and in general lead to a lowering of the performance of the filter. In this section, we will discuss the effects of finite wordlength on the performance of our FIR digital low-pass filter. Three ways in which finite word length affects the performance of FIR digital filters are:

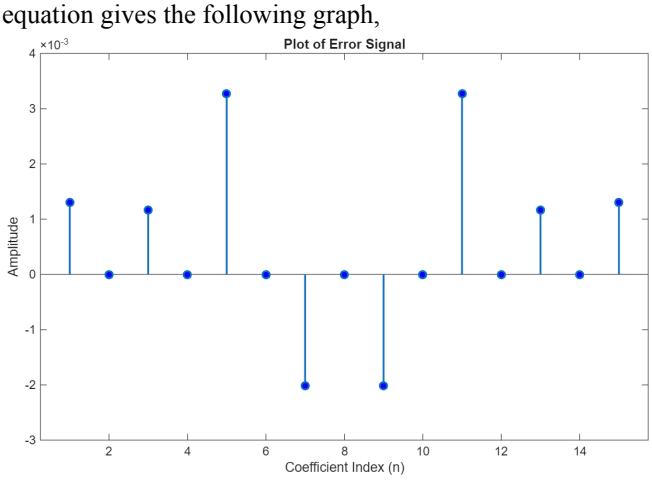
(1) Coefficient Quantization Error

These result from representing filter coefficients with a limited number of bits. In our filter design, the coefficients are 8-bit wide. As all the filter coefficients are < 1, we will use Q7 format to represent the coefficients in binary format, i.e., 1 bit for sign and 7 bits for fraction.

In doing so, however, we automatically introduce an error which causes the frequency response of the finite wordlength filter to deviate from the desired response. We can get valuable insights by analyzing these errors. Now, the unquantized and quantized coefficients, h(n) and $h_q(n)$, respectively, are related as,

$$h_q(n) = h(n) + e(n),\ n=0,1,....,N-1$$

where e(n) is the error between the quantized and unquantized coefficients. Calculating the value of e[n] from the above equation gives the following graph,



(Fig.4. Error between quantized and unquantized coefficients)

To find how much error is actually acceptable so that the filter specifications are satisfied, bounds on the error signal in the frequency domain are made. These bounds could serve as useful guides in determining a suitable coefficient wordlength for a given filter. The most widely used bound is given by,

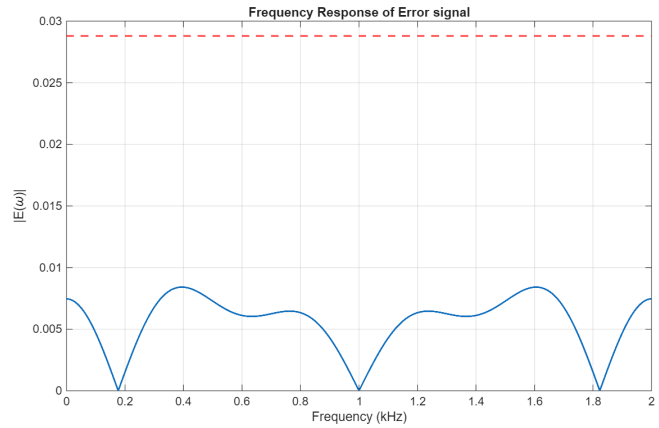
$$|E(\omega)| < 2^{-B}.[(N \log_e N)/3]^{1/2}$$

where B is the number of bits used to represent each coefficient and N is the filter length. Putting the value of B and N in the formula, we get,

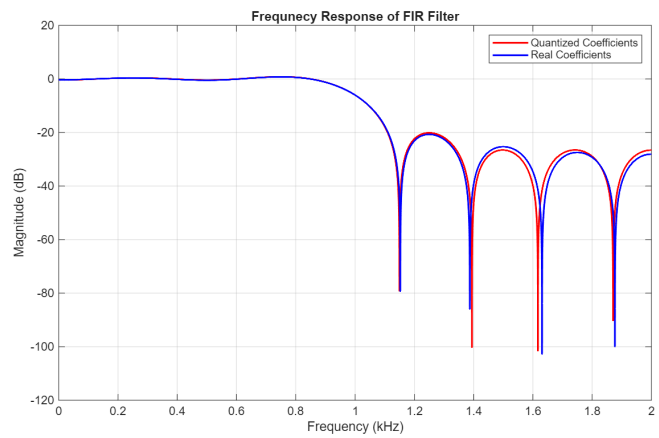
$$|E(\omega)| < 0.0288$$

This means that if the frequency response of the error signal is less than 0.0288, then we can say that the frequency response of the actual filter meets the specification after quantizing the filter coefficients.

The red line in the figure 5 corresponds to a 0.0288 value. We can clearly see that the magnitude of the frequency response is less than 0.0288. The graph in fig. 5. confirms that the error after quantization is not affecting the frequency response of the filter.



(Fig. 5. Frequency Response of error signal)



(Fig. 6. Frequency Response of Quantized Coefficients)

From the above analysis, we can conclude that the coefficient quantization error is not affecting the frequency response of our filter, and we can use the Q7 format to represent filter coefficients.

(2) Arithmetic Overflow

This occurs when partial sums or filter output exceeds the permissible wordlength of the system. Essentially, when an overflow occurs, the output sample will be wrong (normally the sign changes). <u>For our filter design we will scale the input samples such that overflows are minimum.</u>

One solution for the overflow is to saturate the output to the maximum possible value. We will implement this approach in our filter.

(3) Round off errors from quantizing results of arithmetic operation

After each multiplication operation the product formed will be of 14 bits. The product needs to be quantized back to 7 bits before other arithmetic operations are performed. This quantization leads to errors whose effects are similar to those of the ADC noise, but could be more severe. The common way to quantize the result of an arithmetic operation is to truncate the results, that is to retain the most significant higher-order bits and to discard the lower-order bits. Therefore, the product of 14 bits will be quantized by truncating 7 LSB.

Roundoff errors can be minimized by representing all products exactly, with double-length registers, and then rounding the results after obtaining the final sum, that is after obtaining y(n). This approach introduces a smaller error than the alternative approach of rounding each product separately before summing. However, we will avoid this approach because it would degrade the filter's performance, as using double-length adders can slow down processing. Additionally, the reduction in error is not significant enough to justify this performance trade-off.

V. Filter Structure

The FIR filter is characterized by the transfer function, H(z), given by

$$H(z) = \sum_{n=0}^{14} h(n)\, z^{-n}$$

There are mainly two types of realizations: Transversal & Linear Phase. For our filter design, we have used a Linear Phase structure, as we get the advantage of symmetrical filter coefficients. It reduces the computational complexity of filter implementation. Other advantages of using Linear Phase structure are that it takes less area and consumes less power due to a reduced number of multiplication and addition operations. For a linear phase filter, the transfer function can be written as

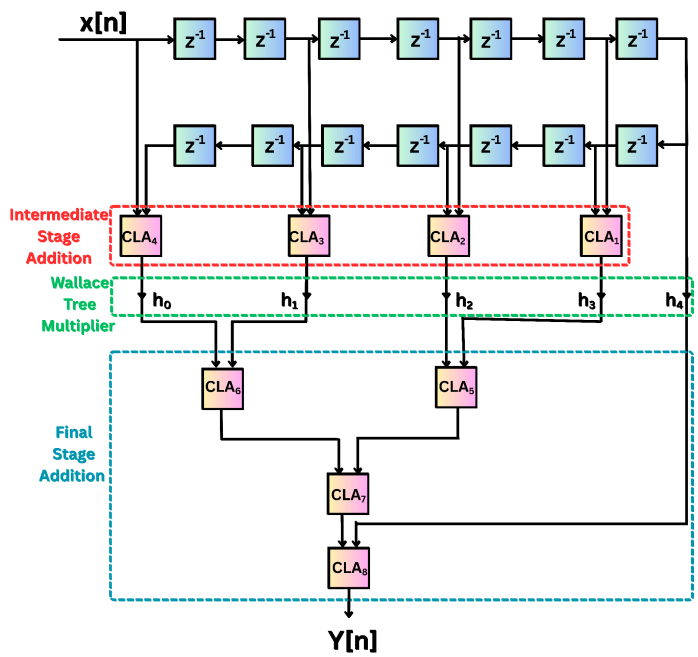
$$H(z) = \sum_{n=0}^{6} h(n)\,[z^{-n} + z^{-(14-n)}] + h(7)z^{-7}$$

For a Linear Phase structure, the computation of each output sample, y(n), requires

- 5 memory locations (ROM) to store 5 coefficients
- 14 RAM locations to store delayed samples of input
- 5 Multiplication Operation
- 8 Addition Operation

In Figure 7, the symbol z^{-1} represents a delay of one sample or unit of time. These delay elements will be implemented by shift registers.

To perform signed multiplication, we will use a signed carry save multiplier because of its simple array structure and easy implementation. The intermediate addition operation is done using Carry Look Ahead (CLA) adder, as it is a fast adder architecture as compared to other adders. The final addition of 5 products is done in three stages using CLA.



(Fig. 7. Schematic representation of the FIR filter)

VI. Conclusion

In this assignment, an 8-bit linear phase FIR low-pass filter was designed and analyzed step by step. The Q7 fixed-point format was shown to preserve the filter's frequency response with minimal quantisation error. The linear-phase structure significantly reduced computational complexity by utilising symmetric coefficients, leading to fewer multipliers and adders and resulting in an area and power efficient realisation. Overall, the designed filter achieved accurate performance, efficient implementation, and suitability for digital signal processing applications. The implementation of the designed filter will be done in part 2 of this assignment.

VII. References

[1] Digital Signal Processing: A Practical Approach; Ifeachor, Jervis

[2] CMOS VLSI Design: A Circuits and Systems Perspective - Book by David Harris and Neil Weste.

[3] Handbook of Digital CMOS Technology,Circuits , and System - Karim Abbas.